Food Delivery System Database Normalization

In this project, the database was normalized up to Third Normal Form (3NF).

- **First Normal Form (1NF)**

In First Normal Form, each table must contain only single values in each field,
and there should be no repeating groups.

For example:

- Customer information is stored in CUSTOMERS
- Customer addresses are stored separately in ADDRESSES
- Restaurant menu items are stored in MENU_ITEMS

This means one customer can have many addresses without repeating customer details,
and one restaurant can have many menu items without repeating restaurant information.

- **Second Normal Form (2NF)**

Second Normal Form removes partial dependency.

This means all non-key attributes must depend on the whole primary key.

For example:

- ORDER_ITEMS stores order_id and item_id with quantity and subtotal
- MENU_ITEMS depends on item_id
- PAYMENTS depends on payment_id

This structure ensures that information such as item price or payment status
is directly related to its own table instead of being duplicated elsewhere.

- **Third Normal Form (3NF)**

Third Normal Form removes transitive dependency.

This means non-key attributes should not depend on other non-key attributes.

Examples:

- Loyalty tier details are stored in LOYALTY_TIERS
- Cuisine details are stored in CUISINE_TYPES
- Vehicle information is stored in VEHICLE_TYPES

By separating these tables, the database avoids repeating the same tier,
cuisine, or vehicle information multiple times.

Relationship Summary

The ER Diagram includes the following major relationships:

- One customer can place many orders
- One customer can have many addresses
- One restaurant can have many menu items
- One order can contain many order items
- One driver can deliver many orders
- One order can have payment details
- One customer can earn many loyalty events

These relationships help connect the system while keeping data organized.

Benefits of Normalization in this System

- Reduces data redundancy
- Prevents duplicate information
- Improves update efficiency
- Makes querying easier
- Improves database consistency
- Supports future expansion